

Computation and Boundary Evidence for Scientific Software Repair

Rajarshi Ghoshal

rajarshi.ghoshal1@gmail.com

PhD applicant research task — Graz University of Technology

Abstract

Repairing scientific software requires understanding the computations a change must preserve. We pre-compute expressions, conditions, definitions, and call boundaries from repository source and expose them through three query endpoints alongside the agent’s ordinary tools — a representation we call computation and boundary evidence (CBE). On 89 SWE-bench Science tasks with three runs per arm (534 verified attempts), CBE and a matched baseline solve 63 and 60 attempts respectively; the confidence interval spans zero. The discipline-level breakdown is more informative: gains concentrate where the repair corrects a calculation, losses where it must preserve an interface. Two case studies trace this asymmetry to a scope gap — locating the relevant computation is necessary but not sufficient for a complete repair.

1 Introduction

Task 103 in SWE-bench Science asks an agent to repair a composite-laminate stiffness calculation in pyNastran. In the reported attempts, expanding the ply iteration without correcting the accumulation formula passes the public reproducer but fails the complete private suite. The example motivates making the computation surrounding a candidate repair directly inspectable, rather than leaving the agent to reconstruct it through text search.

CBE organizes this repository-derived evidence before repair and exposes it through three query endpoints beside the usual shell tools. Public execution observations guide source selection; static extraction links computations, conditions, definitions, and call boundaries. The baseline uses the same model, budget, and containers without this evidence store.

Our research question is whether this additional representation improves complete repair. The contributions are the preparation pipeline, query interface, and replicated comparison. We analyze both successful and unsuccessful changes to understand when the representation helps.

Related work. Bug localization through traceability links recovers connections between artifacts: RAT (Niu et al., 2023a) builds refactoring-aware trace models, ABLoTS (Niu et al., 2023b) tests their replicability across projects, TVR (Niu et al., 2026) extends recovery to automotive requirements via retrieval-augmented generation, and Pan et al. (2026) minimise natural-language test cases under coverage constraints. These approaches link artifacts but do not extract the computational structure of the code itself. Code property graphs (Yamaguchi et al., 2014) unify ASTs, control flow, and data flow for vulnerability analysis; CBE borrows the multi-view graph idea but targets scientific expressions and call boundaries rather than security patterns. AutoMATES (Pyarelal et al., 2020) and GroMEt (ML4AI Team, 2024) extract scientific models from code and equations into intermediate representations; CBE stays closer to the source, indexing expressions and conditions without attempting semantic model assembly. Tree-sitter (Brunsfield et al., 2024) provides the incremental parsing layer. We evaluate on SWE-bench Science (Xu et al., 2026) with DeepSeek V4.1 Flash (DeepSeek, 2026); SWE-bench (Jimenez et al., 2024) and SciCode (Tian et al., 2024) are related benchmarks that target general software and research coding respectively.

2 What CBE provides

The pipeline has three stages (Figure 1): trace, extract, query.

Trace. Run the public `reproduce.py` under an observer. The observer records which functions execute, guiding file selection for extraction. No model calls. Median duration: 70 s of the 1800 s budget.

Extract. Tree-sitter (Brunsfield et al., 2024) parses Python ASTs for scopes, assignments, binary expressions, return anchors, and branches. Joern handles C, C++, Fortran, and Cython where available; explicit fallback otherwise. External calls are labelled with their provider as interface assumptions.

Table 1. CBE evidence records for task 103 (pyNas-tran).

Record	Content
Computation	Scope, expressions, result anchors: accumulation at line 1003 $\rightarrow$ <code>return ABD</code> at 1015.
Condition	<code>len(z0) == len(z1)</code> at line 987.
Definition	Documented public behavior (not execution trace).
Boundary	<code>np.radians(thetad)</code> at line 979: provider <code>numpy</code> , caller-side scope.
Query interface (alongside ordinary shell)	
<code>find</code>	Keyword search over symbols, docstrings, and expressions.
<code>inspect</code>	Structured view: dependencies, conditions, boundaries.
<code>note</code>	Scratchpad for working hypotheses; never gates repair.

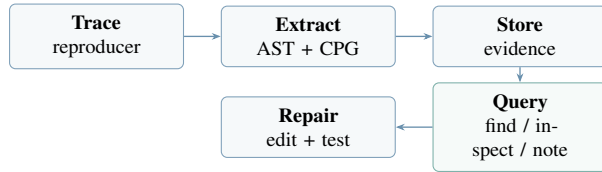


Figure 1. CBE pipeline: trace the reproducer, extract computation structure, store it, then the agent queries and repairs.

Unresolved callees stay `unknown_external`.

Table 1 shows what CBE extracts for task 103 (pyNas-tran) — the opening example. The computation record links the accumulation at line 1003 to its return anchor. The boundary record identifies the `numpy` call and its scope.

Query and repair. The agent accesses the three endpoints alongside its normal shell utilities. `find` searches the indexed structure; `inspect` retrieves computation scopes and boundary details; `note` records a working hypothesis without blocking repair. Inspection refreshes after each edit; the preparation trace stays fixed.

3 Protocol

SWE-bench Science contains 119 tasks across 20 disciplines (Xu et al., 2026). We use 30 tasks for development and evaluate the remaining 89, spanning 18 disciplines. Each task runs three times per arm (534 verified attempts). Both arms share the conditions in Table 2.

A solve is a successful official verifier reward, not merely a passing public reproducer or private-test subset. All reported attempts have verdicts. We

Table 2. Shared evaluation parameters.

Component	Detail
Model	DeepSeek V4.1 Flash (DeepSeek, 2026); temperature 0; <code>PYTHONHASHSEED=0</code> .
Replicates	89 tasks $\times$ 3 runs = 267 per arm; 534 verified.
Budget	1800 s, including preparation.
Verification	Separate container, 1800 s. Private tests not visible.
Network	None.
Baseline	Shell tools + public task material.
CBE	Same + evidence store + three query endpoints.

Table 3. Baseline solves 60, CBE solves 63. CBE generates 5.6% fewer output tokens.

Metric	Baseline	CBE
Scheduled attempts	267	267
Exact repairs / verdicts	60/267	63/267
Exact-repair rate	22.5%	23.6%
Private tests passed	2341/3219	2282/3219
Recorded input (M tokens)	952.4	1126.5
Recorded output (M tokens)	13.56	12.81
Mean recorded work (s)	1203.5	1271.4

Paired task-rate diff: +1.12 pp; 95% bootstrap $[-2.6, +4.9]$ pp; sign $p = 1.00$. All 534 attempts verified (267 per arm); mean work averages runtime across attempts with usage records.

bootstrap per-task rate differences with 20,000 re-samples and a fixed analysis seed; the paired sign test excludes ties.

4 Results

Aggregate comparison. Baseline solves 60 of 267 verified attempts; CBE solves 63 of 267 (Table 3). The paired interval spans zero.

Resource shift. 74 tasks tie, 8 favour CBE, 7 favour baseline. Output drops 5.6% (12.81M vs. 13.56M tokens); input rises 18.3%; work takes 68 s longer.

Hidden partial changes. Even among exact ties, behaviour differs: 29 of 74 differ in private-test pass fractions (baseline passes more on 20, CBE on 9, equal on 45). Among the 15 non-tie tasks, every CBE-favoured task also has CBE-higher private-test coverage, and six of seven baseline-favoured tasks have baseline-higher coverage.

Replication. Baseline and CBE never solve 60 and 59 tasks, respectively. Outcomes are mixed on 18 baseline and 19 CBE tasks; 11 tasks succeed in all three runs for each arm.

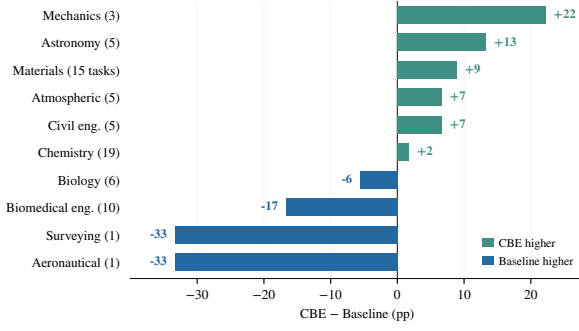


Figure 2. Solve-rate difference (CBE — Baseline) for the ten disciplines with non-zero change. Task counts in parentheses. Eight disciplines with identical solve rates are omitted.

5 Where the difference concentrates

The aggregate comparison is inconclusive; the task-level breakdown reveals where the differences concentrate.

Figure 2 shows gains in six disciplines and losses in four. Mechanics’ gain comes entirely from task 103, while the materials-science gain spans DeePTB, deepmd-kit, and dpdata. Biology’s net loss comes from biopython. Each of the two largest negative differences represents one task. These groups locate where the difference concentrates.

Task demands and representation scope. The benchmark distinguishes *Issue-driven*, *Expert-exploratory*, and *Engineering-integration* tasks (Xu et al., 2026). These describe how a problem is posed, not its scientific discipline or the obligations of its eventual patch. A known defect still requires regression preservation; investigating an anomaly requires comparing possible explanations; integration requires checking behaviour across modules. Computation and call records are relevant to all three; whether they suffice depends on the task. Our analysis lacks per-task paradigm annotations, so we report no subgroup rates.

6 Case studies

The reproducer guides CBE toward code exercised by the task, but the evidence schema is the same for all tasks. The repair agent still determines whether the extracted records explain the discrepancy and cover the behaviours a change must preserve. The following two cases examine this gap.

Task 103 (pyNastran). Figure 3 separates incomplete fixes from complete repairs. All baseline attempts pass the public check but only two of

four private tests. The two successful CBE patches change both expansion and accumulation. Localization alone is not sufficient; the repair must also be complete.

Task 022 (nilearn). The successful baseline patches preserve the existing nearest-neighbour path while adding label aggregation. Two CBE patches alter argument handling and pass 11/12 private tests; the third passes 4/12. Reaching the aggregation code broke the existing interface.

7 Discussion

The two cases illustrate a pattern visible across the discipline split. When the repair obligation is correcting a calculation, CBE’s evidence directly exposes the relevant scope: in task 103, the agent sees the accumulation loop, its return anchor, and the conditions guarding it, and two of three runs produce complete patches. When the obligation is preserving an existing interface, the same evidence points the agent toward internal computations rather than the function contract it must keep stable: in task 022, the agent reaches the aggregation code but alters the argument path instead of preserving backward compatibility. The discipline-level gains (mechanics, astronomy, materials) concentrate in repositories whose failing tests check numerical outputs; the losses (biomedical, biology) concentrate where the test suite checks API behaviour. The representation helps when what it indexes matches what the repair must preserve, and misleads when it does not.

8 Limitations and conclusion

CBE indexes repository evidence by task but uses the same evidence schema across repair types. Definitions and predicates are semantic cues, not specifications; the agent still forms hypotheses and checks edit implications across the workflow.

The public trace covers observed execution; extraction fallbacks and unresolved calls limit structural coverage. External providers are interface assumptions. Three replicates are insufficient for generalization claims; no ablation isolates individual components; most tasks are Python; one model and scaffold.

CBE yields similar repair rates with a resource trade-off. Whether the evidence covers the repair obligation depends on the task; extending it to interface and compatibility requirements is the next

Task / repository	Discipline	Baseline			CBE			Solved / verified
		Run 1	Run 2	Run 3	Run 1	Run 2	Run 3	
042 sunpy	Astronomy	15/16	15/16	6/16	16/16	7/16	16/16	0/3 → 2/3
074 DeePTB	Materials	9/10	9/10	2/10	10/10	10/10	9/10	0/3 → 2/3
103 pyNastran	Mechanics	2/4	2/4	2/4	4/4	4/4	2/4	0/3 → 2/3
067 xclim	Atmospheric	6/6	6/6	3/6	6/6	6/6	6/6	2/3 → 3/3
020 deepmd-kit	Materials	10/12	10/12	11/12	10/12	12/12	10/12	0/3 → 1/3
084 dpdata	Materials	5/7	5/7	7/7	7/7	5/7	7/7	1/3 → 2/3
101 PyBaMM	Chemistry	3/10	3/10	5/10	3/10	10/10	3/10	0/3 → 1/3
116 Pynite	Civil eng.	85/93	85/93	85/93	85/93	93/93	85/93	0/3 → 1/3
029 biopython	Biology	29/29	14/29	27/29	14/29	14/29	14/29	1/3 → 0/3
053 MONAI	Biomedical	2/6	6/6	2/6	5/6	5/6	2/6	1/3 → 0/3
087 dipy	Biomedical	5/5	0/5	1/5	0/5	2/5	2/5	1/3 → 0/3
093 MONAI	Biomedical	4/4	2/4	2/4	2/4	2/4	2/4	1/3 → 0/3
098 satpy	Surveying	11/11	8/11	11/11	8/11	11/11	8/11	2/3 → 1/3
056 poliastro	Aeronautical	10/10	10/10	10/10	10/10	10/10	1/10	3/3 → 2/3
022 nilearn	Biomedical	12/12	12/12	10/12	4/12	11/12	11/12	2/3 → 0/3

Cell: private tests passed / collected. Outlined + bold: official repair. Dash: no verdict.

Figure 3. Per-run private-test pass fractions for every task whose solve rate differs between arms. Outlined bold cells are official solves; bold task labels are the two cases below.

Table 4. Comparison of generated patches for divergent case studies.

Task	Baseline	CBE	Insight
103 pyNas-tran	Expand ply iteration; keep original B/D accumulation. All pass public.	Two fix both expansion and B/D; third keeps old accumulation.	The public check passes both partial and complete repairs.
022 nilearn	Successful patches add label aggregation while preserving the nearest-neighbour path.	Two alter argument handling (11/12 private); the third passes 4/12.	Reaching the aggregation code did not preserve the interface.

step.

9 AI Assistance

ChatGPT and Oh My Pi assisted with coding, debugging, analysis, and plotting. Code, receipts, and per-task outcomes: <https://github.com/rajarshighoshal/computation-boundary-evidence>.

References

- Max Brunsfeld et al. Tree-sitter: An incremental parsing system for programming tools, 2024. URL <https://tree-sitter.github.io>.
- DeepSeek. DeepSeek V4.1 Flash: Model card and API documentation, 2026. URL <https://api-docs.deepseek.com>.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models

resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024.

ML4AI Team. GroMET: A unified intermediate representation for scientific models and code. *DARPA ASKEM / SKEMA Consortium*, 2024.

Feifei Niu, Wesley K. G. Assunção, Liguang Huang, Christoph Mayr-Dorn, Jidong Ge, Bin Luo, and Alexander Egyed. RAT: A refactoring-aware traceability model for bug localization. In *Proc. 45th International Conference on Software Engineering (ICSE)*, 2023a.

Feifei Niu, Christoph Mayr-Dorn, Wesley K. G. Assunção, Liguang Huang, Jidong Ge, Bin Luo, and Alexander Egyed. The ABLoTS approach for bug localization: Is it replicable and generalizable? In *Proc. 20th International Conference on Mining Software Repositories (MSR)*, pages 576–587, 2023b. doi: 10.1109/MSR59073.2023.00083.

- Feifei Niu, Rongqi Pan, Lionel C. Briand, and Hanyang Hu. TVR: Automotive system requirement traceability validation and recovery through retrieval-augmented generation. *Empirical Software Engineering*, 2026.
- Rongqi Pan, Feifei Niu, Lionel C. Briand, and Hanyang Hu. Requirements coverage-guided minimization for natural language test cases. *ACM Transactions on Software Engineering and Methodology*, 2026.
- Adarsh Pyarelal et al. AutoMATES: Automated model assembly from text, equations, and software. *arXiv preprint arXiv:2001.07295*, 2020.
- Minyang Tian, Luyu Gao, Shizhuo Dylan Zhang, et al. SciCode: A research coding benchmark curated by scientists. *arXiv preprint arXiv:2407.13168*, 2024.
- Zhipeng Xu, Jiahao Lu, Yining Zheng, Yuxin Wang, and Xipeng Qiu. SWE-bench Science: Can coding agents resolve engineering tasks in science? *arXiv preprint arXiv:2608.19799v2*, 2026.
- Fabian Yamaguchi, Nico Golde, Daniel Arp, and Konrad Rieck. Modeling and discovering vulnerabilities with code property graphs. In *IEEE Symposium on Security and Privacy*, 2014. doi: 10.1109/SP.2014.44.